PYTHON FOR BEGINNERS AND PROFESSIONALS

SESSION 05

Functions in Python

What is a Function?

A function is a reusable block of code that performs a specific task. Instead of writing the same logic again and again, you write it once inside a function and call it whenever you need it.

This follows the **DRY** principle - **Don't Repeat Yourself**. It is one of the most important principles in programming.

Functions also make your code easier to read, easier to debug, and easier to maintain. If the logic needs to change, you change it in one place not everywhere.

Example

```
# Without functions — repetitive
print("Welcome, Rahul!")
print("You have 3 new messages.")
print("Welcome, Priya!")
print("You have 3 new messages.")

# With functions — clean and reusable
def greet(name):
    print(f"Welcome, {name}!")
    print("You have 3 new messages.")

greet("Rahul")
greet("Priya")
```

Defining and Calling a Function

You define a function using the ***def*** keyword followed by the function name and parentheses. You call it by writing its name followed by parentheses.

```
# Defining a function
def function_name():
    # code goes here

# Calling a function
function_name()
```

Practice Questions on Functions

Q35. Write a function that ask a number from user and prints if that number is odd or even.

Q36. Write a function that print all the factors of a number entered by user.

Parameters and Arguments

A ***parameter*** is a variable listed inside the function definition. An ***argument*** is the actual value you pass when calling the function. Parameters make functions flexible - the same function can work with different data every time.

```
def greet(name):              # name is the parameter
    print(f"Hello, {name}!")

greet("Rahul")                # "Rahul" is the argument
greet("Priya")                # "Priya" is the argument
```

Practice Questions on Parameters and Arguments

Q37. Write a function called *add* that takes two numbers as parameters and prints their sum.

Q38. Write a function called *rectangle_area* that takes *length* and *breadth* as parameters and prints the area.

Q39. Write a function called *find_max* that takes three numbers as parameters and prints the largest one.

Q40. Write a function called *discount_price* that takes *original_price* and *discount_percent* as parameters and prints the final price after discount.

return Statement

The *return* statement sends a value back to whoever called the function.
Without *return*, a function just does something but gives nothing back.
With *return*, you can use the result in the rest of your program.

```
# Without return — result is lost
def add(a, b):
    print(a + b)    # prints but gives nothing back

# With return — result can be stored and used
def add(a, b):
    return a + b

result = add(10, 5)
print(result)           # 15
print(add(3, 7) * 2)    # 20
```

Note: Once Python hits a return statement, the function stops immediately. Any code after return inside the function will never run.

Practice Questions on Return Statements

Q41. Write a function called *square* that takes a number and returns its square. Store the result and print it.

Q42. Write a function called *min_of_three* that takes three numbers and returns the smallest without using any built-in function.

Q43. Write a function called *absolute_value* that takes a number and returns its absolute value *without* using the built-in *abs()* function.

Default Arguments

A default argument is a value that a parameter takes automatically if no argument is passed for it when calling the function. It makes certain parameters optional.

```
def greet(name, message="Good Morning"):
    print(f"{message}, {name}!")

greet("Rahul")                      # Good Morning, Rahul!
greet("Priya", "Good Evening")      # Good Evening, Priya!
```

Watch Out: Parameters with default values must always come after parameters without default values.
def greet(message="Hi", name) will give you a *SyntaxError*.

Keyword Arguments

Keyword arguments let you pass values to a function by explicitly naming the parameter. This means you can pass them in any order you are no longer dependent on position.

```
def introduce(name, age, city):
    print(f"{name} is {age} years old from {city}.")

# Positional — order matters
introduce("Rahul", 21, "Delhi")

# Keyword — order does not matter
introduce(age=21, city="Delhi", name="Rahul")
introduce(city="Mumbai", name="Priya", age=19)
```

Quick Tip: Keyword arguments make your function calls much more readable, especially when a function has many parameters. It is always clear what value is going where.

Functions Calling Other Functions

A function can call another function inside it. This is how you break a big problem into smaller pieces each function does one job and they work together.

```
def square(n):
    return n * n

def sum_of_squares(a, b):
    return square(a) + square(b)    # calling square() inside

print(sum_of_squares(3, 4))         # 9 + 16 = 25
```

Quick Tip: Each function should do one thing and do it well. If you find a function doing too many things, split it into smaller functions and call them together.

Local vs Global

Scope refers to where a variable is accessible in your code. A variable created inside a function is local it only exists inside that function. A variable created outside all functions is global it can be accessed from anywhere.

```
# Global variable
name = "Rahul"

def greet():
    # Local variable
    message = "Hello"
    print(f"{message}, {name}!")    # can read global variable

greet()                 # Hello, Rahul!
print(message)          # NameError — message doesn't exist outside
```

The global Keyword

If you want to modify a global variable from inside a function, you must explicitly tell Python using the *global* keyword.

```
count = 0

def increment():
    global count        # tells Python to use the global count
    count += 1

increment()
increment()
increment()
print(count)            # 3
```

Lambda Functions

A lambda function is a small *anonymous* function written in a single line. It is useful when you need a simple function for a short period and do not want to formally define it with *def*.

```
# Normal function
def square(n):
    return n * n

# Same thing as a lambda
square = lambda n: n * n

print(square(5))    # 25
```

Practice Questions on LAMBDA Functions

Q44. Write a lambda function that takes a number and returns its cube. Store it in a variable and call it.

Q45. Write a lambda function that takes a number and returns "Positive", or "Negative".

Practice Questions Functions (Assignments)

Q46. Write a function *fizzbuzz(n)* that takes a single number and prints *"Fizz"* if it's divisible by 3, *"Buzz"* if it's divisible by 5, *"FizzBuzz"* if it's divisible by both, otherwise print the number itself.

```
# Example 1
fizzbuzz(9)
# Output
Fizz

# Example 2
fizzbuzz(20)
# Output
Buzz

# Example 3
fizzbuzz(15)
# Output
FizzBuzz

# Example 4
fizzbuzz(7)
# Output
7
```

Practice Questions Functions (Assignments)

Q47. Write a function *power(base, exp)* that returns base raised to exp using a loop - no ** operator or *pow()* allowed.

Q48. Write a function *tax_calculator(income)* that takes annual income and returns the tax amount based on these slabs:

- Up to 2,50,000 → No tax
- 2,50,001 to 5,00,000 → 5%
- 5,00,001 to 10,00,000 → 20%
- Above 10,00,000 → 30%